Git, GitHub & Deployment Interview Knowledge

Version control, secrets, frontend/backend deployment and production troubleshooting

Purpose
Git/GitHub, secrets, React/Next.js + FastAPI deployment, Neon PostgreSQL, Qdrant Cloud and Render operations.

Audience: Final-year engineering students and fresher candidates targeting Python, software development, ML, AI engineering and GenAI roles.

RAG design: Content is explicit and retrieval-friendly so chunks can stand alone. Source identifiers, definitions, examples, troubleshooting and interview reasoning are intentionally separated.

Table of Contents

1. 1. Git mental model
2. 2. Branches, merges and conflicts
3. 3. GitHub and README quality
4. 4. Secrets and environment variables
5. 5. Frontend/backend architecture
6. 6. Render FastAPI deployment
7. 7. Neon PostgreSQL and migrations
8. 8. Qdrant Cloud deployment
9. 9. Production troubleshooting
10. 10. End-to-end deployment verification

1. Git mental model

Git is distributed version control. The working tree contains checked-out files; the staging area/index records what will enter the next commit; commits store snapshots and history.

A focused commit is easier to review and revert. A commit is local until pushed to a remote such as GitHub.

```
git status
git add .
git commit -m "Add RAG evaluation"
git fetch origin
git pull --rebase origin main
git push origin feature/rag-evaluation
```

2. Branches, merges and conflicts

A branch is a movable reference to a commit. Merge combines histories; rebase replays commits onto a new base and rewrites commit IDs. Rebase is useful for local cleanup but should be used carefully on shared history.

When conflicts occur, resolve the intended final code, remove conflict markers, stage the result and continue the merge/rebase.

```
git switch -c feature/api-health
# edit files
git add .
git commit -m "Add health endpoint"
git switch main
git pull --ff-only
git merge --no-ff feature/api-health
```

3. GitHub and README quality

GitHub hosts Git repositories and collaboration features such as pull requests and issues. A useful README tells a new developer what the system does, how to run it, configure it safely, test it and deploy it.

For an AI interview system, document architecture, RAG flow, environment variable names without values, ingestion commands, API endpoints, tests and known limitations.

- Never paste API keys or database credentials into README examples.

4. Secrets and environment variables

Use environment variables or a secret manager for database URLs, model API keys, Qdrant credentials and JWT signing secrets. `.env.example` can document variable names with placeholders.

If a secret is committed, deleting the file in a later commit is not enough. Rotate/revoke the credential and consider history cleanup. Client-side bundles must not receive server-only secrets.

```
# .env.example only
DATABASE_URL=postgresql://...
GEMINI_API_KEY=replace-me
QDRANT_URL=https://...
QDRANT_API_KEY=replace-me
JWT_SECRET=replace-me
FRONTEND_ORIGIN=https://frontend.example.com
```

5. Frontend/backend architecture

A typical deployment is browser → React/Next.js frontend → FastAPI backend → PostgreSQL and Qdrant Cloud, with the backend calling the model provider. Provider secrets should remain on the server.

The frontend must use the deployed API base URL. The backend must allow the exact frontend origin through CORS. A green frontend build does not prove the integration works.

Component	Role	Important config
React/Next.js	UI/client	API base URL
FastAPI	Auth/business/RAG	PORT, secrets, CORS
Neon PostgreSQL	Relational data	DATABASE_URL
Qdrant Cloud	Vector search	URL/API key
Gemini API	Embeddings/generation	API key/model

6. Render FastAPI deployment

Render’s current FastAPI deployment documentation shows a Python build command such as `pip install -r requirements.txt` and a start command using Uvicorn on `0.0.0.0` and the platform-provided `$PORT`. Exact module/app names must match your repository.

Render health checks can confirm service readiness. An HTTP health endpoint can be used for application-level readiness. Startup/import errors and missing environment variables should be diagnosed from logs first.

```
Build Command:
pip install -r requirements.txt

Start Command:
uvicorn main:app --host 0.0.0.0 --port $PORT

@app.get("/health")
def health():
    return {"status": "ok"}
```

7. Neon PostgreSQL and migrations

Hosted PostgreSQL should be configured using the provider’s connection string and current pooling guidance. Treat the connection string as a secret.

Schema migrations should be controlled and repeatable. In SQLAlchemy projects, Alembic is commonly used. `create_all()` is not a substitute for a migration history when schemas evolve.

```
# Example migration workflow
alembic revision --autogenerate -m "add interview_attempts"
alembic upgrade head

# Deployment principle:
# build -> controlled migration -> start
```

8. Qdrant Cloud deployment

Keep collection name, embedding-model identity, vector dimensions and document versioning explicit. If the embedding model changes, plan a re-index.

Stable point IDs make repeated ingestion idempotent. Check collection state before uploading again; blindly inserting can create duplicates.

```
QDRANT_URL = os.environ["QDRANT_URL"]
QDRANT_API_KEY = os.environ["QDRANT_API_KEY"]
COLLECTION = "interview_knowledge_v2"
EMBEDDING_MODEL = os.environ["EMBEDDING_MODEL"]
```

9. Production troubleshooting

Build failure: inspect Python version, requirements and import paths. Startup failure: inspect traceback and missing configuration. API failure: check frontend URL, CORS and backend routes. Database failure: verify URL, TLS/pooling, credentials and migrations. RAG failure: inspect collection, vector compatibility, ingestion state and metadata filters.

Cold starts or startup initialization can make the first request slower than warm requests. Measure before concluding the model or database is inherently slow.

Symptom	Checks
Build fails	Runtime version, requirements, imports
App exits	Startup traceback, env vars
Frontend calls localhost	Production API URL
CORS error	Exact origin/preflight
DB connection error	URL, SSL, pool, credentials
RAG empty	Collection, vectors, dimensions, filters
Stale RAG	Document version/re-index
Health failure	Port/path/readiness

10. End-to-end deployment verification

Verify the deployed frontend in a clean session, confirm the API base URL, call the health endpoint, test authentication, exercise one database path and run a known RAG question with expected source metadata. Check logs after each boundary.

Deployment verification is a chain of evidence, not a single green build badge. Record the deployed commit/version so a regression can be traced or rolled back.

- Do not test only from a developer machine with local environment variables.
- Do not expose provider secrets to browser JavaScript.
- Do not repeatedly re-index production just because a query failed.

Interview question

Git versus GitHub?

Strong answer: Git is the version-control system; GitHub is a hosted collaboration platform built around Git repositories and additional workflows.

Follow-up: What can Git do without GitHub?

Interview question

Merge versus rebase?

Strong answer: Merge combines histories and preserves branch structure. Rebase replays commits onto a new base and rewrites commit IDs. It is useful for local cleanup but risky when rewriting history others depend on.

Follow-up: Why can force-push be dangerous?

Interview question

Why are secrets unsafe in Git history?

Strong answer: Git history is durable and copied. A secret committed once may remain in old commits, forks or caches. Rotate it immediately and clean history where appropriate.

Follow-up: What belongs in `.env.example`?

Interview question

Why can deployment be green while the app is broken?

Strong answer: Build success proves compilation/installation/startup steps passed; runtime configuration, CORS, database connectivity, migrations, external services and frontend URLs can still be wrong.

Follow-up: What is the first debugging source?

Interview question

How would you debug a production RAG failure?

Strong answer: Separate ingestion, retrieval and generation. Check the production Qdrant connection, collection and vector compatibility, run a direct safe retrieval query, inspect payloads and only then

inspect prompt/model behavior.

Follow-up: Why separate the stages?

Interview question

Why use migrations?

Strong answer: They provide a controlled, versioned history of schema changes that can be applied consistently across environments.

Follow-up: Why is `create_all()` insufficient for evolving production schemas?

Scenario: frontend build succeeds but requests fail

Situation: Inspect browser network calls. If the URL is localhost, fix frontend configuration. If the URL is correct but blocked, inspect CORS. If it returns 404/500, inspect backend routing and logs.

Reasoning: A successful frontend build is not an integration test.

Scenario: login fails only in production

Situation: Compare request URL, CORS, environment variables, user lookup, password hashing and JWT configuration. Use logs without exposing secrets.

Reasoning: Disabling authentication as a “fix”.

Scenario: RAG works locally but not in production

Situation: Verify Qdrant URL/key, collection name, embedding model/dimensions, ingestion state, metadata filters and network access. Run a direct diagnostic query with a known question.

Reasoning: Re-uploading documents without checking collection state.

Revision checklist

- Explain working tree/staging/commit.
- Use branches and resolve conflicts.
- Write a useful README.
- Protect and rotate secrets.
- Explain frontend/backend separation.
- Know Render FastAPI start/health concepts.
- Understand PostgreSQL migrations.
- Understand Qdrant deployment compatibility.
- Debug CORS/API URL/logs systematically.
- Verify production end to end.

Reference notes

Primary documentation used for version-sensitive guidance. Verify exact package/API behavior against the versions installed in your project.

https://git-scm.com/doc
https://docs.github.com/
https://render.com/docs/deploy-fastapi
https://render.com/docs/health-checks
https://neon.com/docs/
https://qdrant.tech/documentation/

End of knowledge base document